Submission Preparation Draft and Evidence Report

Course activity: EGN475 W1A1 — Prototyping Environment
Project: Café Kouis Flask Prototype
Student: Kouis Younes
Prepared with disclosed automated assistance

> **Academic-integrity note.** This is a preparation draft, not a representation that the student independently wrote, tested, or explained the program. Manus created and verified project materials in an Ubuntu sandbox and uploaded them to GitHub through the signed-in browser. The student must review the code, run it in the required environment, rewrite the personal answers below, and disclose assistance under course policy.

1. Completion status

Requirement	Status	Evidence / note
Python virtual environment with Flask	Completed in Manus sandbox	Python 3.12.3 and Flask 3.1.3; this is not the student's Windows environment
`.gitignore` for `.venv/` and `__pycache__/`	Completed	Both entries are in the public repository's `.gitignore`
Flask `app.py` with `/` and `/about`	Completed and verified	Tests run against a fresh clone of the public repository; both routes returned HTTP 200
At least three meaningful Git commits	Completed publicly	The five core project commits below appear on GitHub `main`, followed by documentation updates; see Section 4
Actual browser-testing screenshots	Completed	Captured with Chromium against the running Flask application; see Section 3
VS Code screenshot	Not available	The Windows desktop and VS Code were not accessible; no screenshot was fabricated
GitHub Copilot interaction	Not available	Copilot Chat was not accessed; no interaction is claimed
Public GitHub repository	Completed	github.com/youneskouisaz-dot/egn475-cafe-kouis
Assignment instructions / exact four questions	Not supplied here	The answers below are draft topics, not confirmed to match the instructor's exact wording

2. Prototype implementation

The public repository contains the Flask source, templates, static styling, tests, setup instructions, report, and evidence files:

File / folder	Responsibility
`app.py`	Creates the Flask application and maps URL paths to route functions
`templates/index.html`	Home page rendered for `/`
`templates/about.html`	About page rendered for `/about`
`static/style.css`	Shared styling for the pages
`tests/test_app.py`	Four automated route and navigation tests
`requirements.txt`	Declares the Flask dependency
`.gitignore`	Excludes the virtual environment and Python cache files
`Q_AND_A_DRAFTS.md`	Four personal-explanation drafts to revise
`AI_INTERACTION_LOG.md`	Records assistance, attribution, and remaining work

The route logic is intentionally small:

```
@app.get("/")
def home():
    return render_template("index.html", page_title="Café Kouis | Home")

@app.get("/about")
def about():
    return render_template("about.html", page_title="Café Kouis | About")
```

The route decorators connect URL paths to Python functions. Each function calls `render_template`, which loads a matching template from `templates/` and returns the rendered page. The templates use Flask's `url_for` helper for navigation and static-file links.

3. Testing evidence

Automated tests and public-repository verification

The test suite uses Python's built-in `unittest` framework. It covers:

1. `/` returns HTTP 200 and expected home-page content.
2. `/about` returns HTTP 200 and expected about-page content.
3. The home page links to the about page.
4. An unknown URL returns HTTP 404.

Verified result: I cloned the public GitHub repository to a fresh temporary directory and ran the tests from that clone: 4 tests ran and passed (`OK`). A separate Flask test-client smoke check against the cloned source returned HTTP 200 for both `/` and `/about`. The detailed initial test output is in `automated-test-output.txt` at the repository root.

Browser captures

The application was started on `127.0.0.1:5050` in the Manus sandbox. Both routes returned HTTP 200 over HTTP, and Chromium captured the actual rendered pages below. These are browser screenshots, not screenshots of VS Code or the user's Windows desktop.

Figure 1. Actual Chromium capture of the `/` home page.

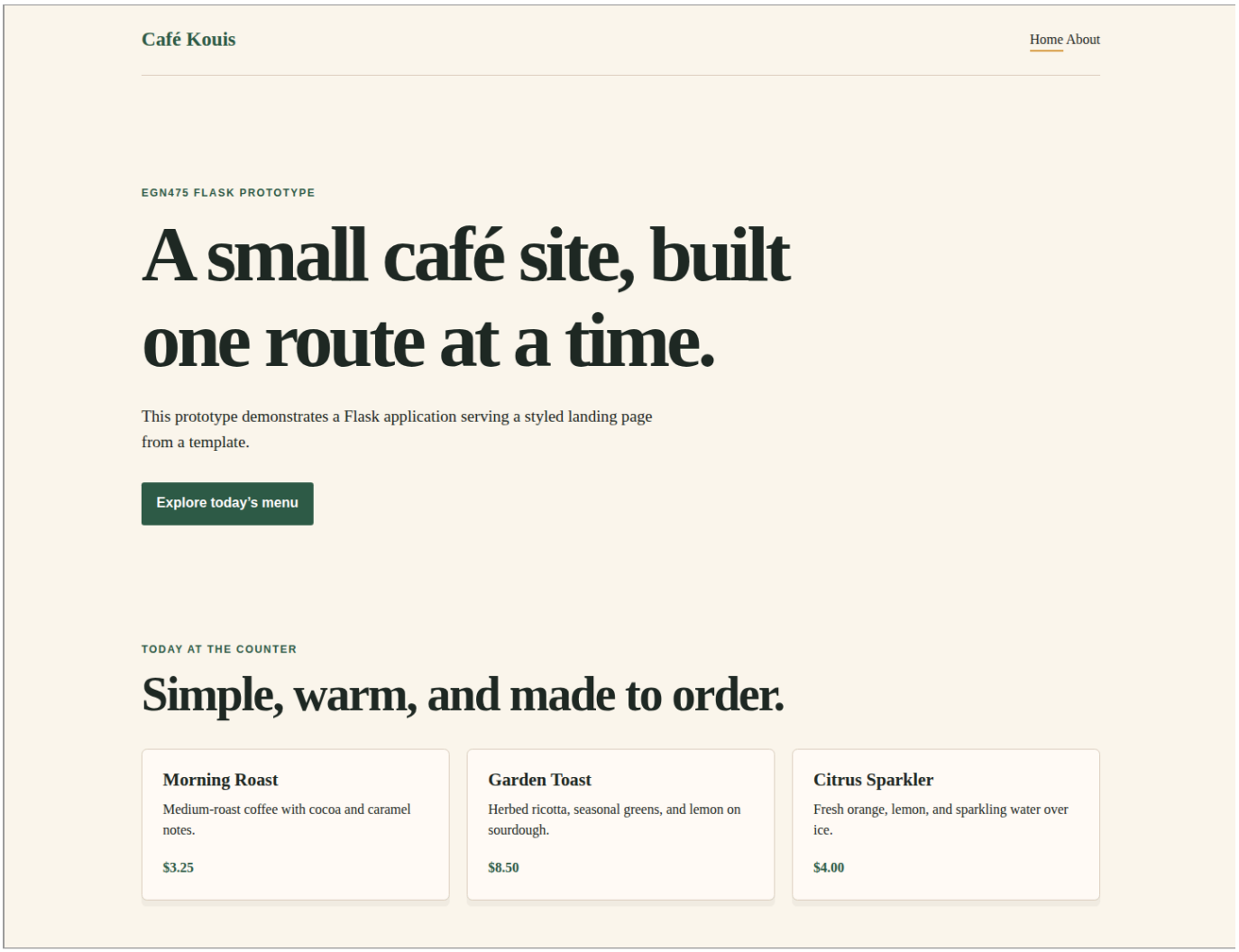
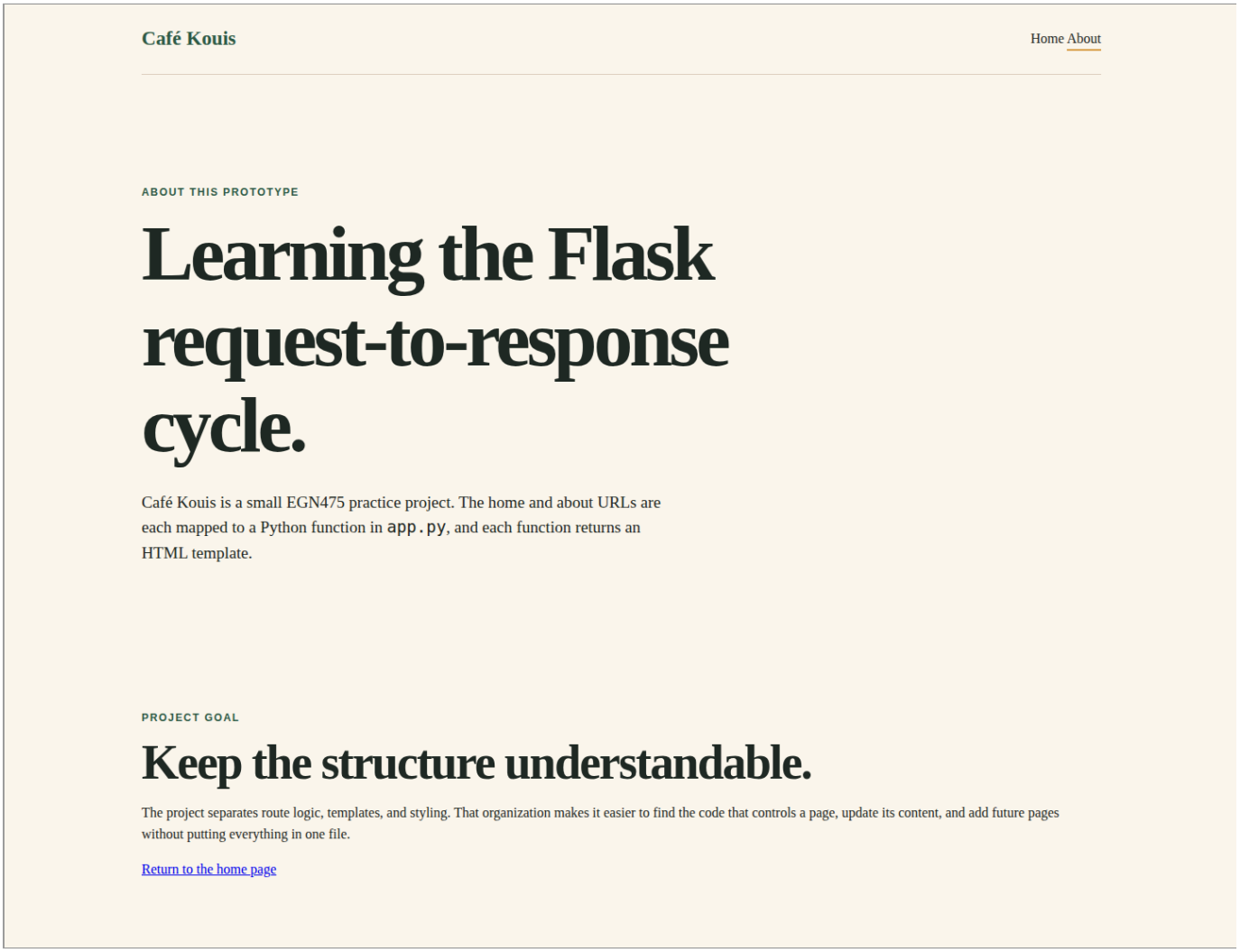


Figure 2. Actual Chromium capture of the `/about` page.



4. Public GitHub repository and commit history

Public repository: https://github.com/youneskouisaz-dot/egn475-cafe-kouis

The repository was created after the user signed in to GitHub in the browser. Manus uploaded the source and evidence using GitHub's web interface. The following five core project commits on `main` satisfy the three-commit requirement; additional documentation and README maintenance commits followed them.

Commit	Message	Change
`42a0666`	`Create README.md`	Added setup, run, and test instructions
`af5cd28`	`Add Flask source and assignment evidence`	Added the Flask app, dependency and ignore files, drafts, report, test output, and browser captures
`70b9fe4`	`Add home and about page templates`	Added the two HTML templates under `templates/`
`a8e7b6e`	`Add shared cafe page styling`	Added the stylesheet under `static/`
`135c5bc`	`Add automated Flask route tests`	Added the four tests under `tests/`
`06b76c2`	`Update verified report and AI log`	Replaced the initial status with the verified public repository, commit history, and clone-based test result
`91e91c3`	`Improve README project structure and setup guide`	Updated the README with the project tree and Windows setup instructions

The five core project commits exceed the requested minimum. GitHub attributes the web-created commits to the signed-in account; Manus prepared/uploaded files and submitted the commit actions. Do not represent them as commits personally authored by the student. A separate local Git history exists in the sandbox but was **not pushed** to GitHub. The report and interaction log were refreshed after the README maintenance commit; that final documentation upload is also visible in the live GitHub history.

5. Personal-explanation drafts

> **Revise before submitting.** The instructor’s exact four questions were not supplied. These are draft answers for common questions about the prototype, not confirmed copies of the assignment prompts. Rewrite them in your own words and be prepared to explain the code.

Draft 1 — Why use a virtual environment?

A virtual environment keeps a project's Python packages separate from packages used by other projects. Here, Flask is installed in `.venv`, and `requirements.txt` records the project's dependency so it can be installed again. This helps avoid changing the system-wide Python installation and makes setup more repeatable.

Draft 2 — What does `.gitignore` do?

The `.gitignore` file tells Git which files it should leave untracked. This project ignores `.venv/`, which contains a local environment and installed packages, and `__pycache__/`, which Python creates for cache files. Ignoring these generated or machine-specific files keeps the repository focused on source and documentation.

Draft 3 — How do the Flask routes work?

The decorators above the Python functions connect URLs to those functions. `@app.get("/")` maps the home URL to `home()`, and `@app.get("/about")` maps the about URL to `about()`. Each function calls `render_template`, which loads a corresponding HTML file from `templates/` and returns it as the response.

Draft 4 — Why make separate Git commits?

Separate commits make a project's history easier to review. Each commit records a meaningful step, such as adding the Flask source, templates, styling, or tests. A clear history helps someone understand how the project changed and makes it easier to locate or undo a change if needed.

6. Truthful AI interaction and environment note

Manus wrote and tested the starter project, prepared the report and draft explanations, and uploaded the files after the user signed in to GitHub. The public repository, core project commits, and later documentation commits are real and verified. The local virtual environment and test execution occurred in an Ubuntu Manus sandbox (Python 3.12.3), not on the described Windows 11 computer. GitHub Copilot Chat and VS Code were not accessed, so there are no Copilot interactions or VS Code screenshots in this report. See `AI_INTERACTION_LOG.md` for the full record.

7. Student actions still required

1. Review every source file and make sure you can explain how the routes, templates, tests, and Git ignore rules work.

2. On the Windows 11 computer, create and activate a local environment, install the dependencies, and run the application and tests:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
flask --app app run
```

1. Use the exact assignment sheet to rewrite the four answers in your own words; the instructor’s prompts were not available here.
2. Capture genuine Windows/VS Code or Copilot evidence yourself if it is specifically required. None has been fabricated or represented as completed.
3. Follow course rules for disclosing/citing AI assistance and check that GitHub attribution is represented accurately.

Appendix: Verification commands

```
.venv/bin/python -m unittest discover -s tests -v
curl http://127.0.0.1:5050/
curl http://127.0.0.1:5050/about
git log --oneline --decorate
git check-ignore -v .venv/bin/python __pycache__/app.cpython-312.pyc
```